

React Learning Notes

Study notes from the `reactpractice` project. Each numbered folder is a separate Vite + React mini-app.

Stack: React 19 · Vite · JSX · CSS Modules · Tailwind CSS v4 · lucide-react

Table of Contents

1. [Project Setup](#)
 2. [JSX Basics](#) — `01-folder`
 3. [Components](#) — `02-components`
 4. [Props & Lists](#) — `03-props`
 5. [CSS Modules](#) — `05-css`
 6. [Tailwind CSS](#) — `06-tailwind`
 7. [UI Project](#) — `07-ui-project`
 8. [Event Handlers](#) — `08-functions`
 9. [Quick Reference](#)
-

1. Project Setup

Every module follows the same Vite + React structure:

```
folder-name/
├─ index.html      # Entry HTML (has <div id="root">)
├─ src/
│   ├─ main.jsx    # Renders App into the DOM
│   ├─ App.jsx     # Root component
│   └─ index.css   # Global styles
├─ package.json
└─ vite.config.js
```

How React mounts to the page

`main.jsx` is the entry point. It finds the `#root` div in `index.html` and renders your `App` component inside it.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Part	What it does
<code>createRoot</code>	Creates a React root attached to a DOM element
<code>render</code>	Puts your component tree into that root
<code>StrictMode</code>	Dev-only checks for common React mistakes

Run any module

```
cd 01-folder    # (or any module folder)
npm install
npm run dev
```

2. JSX Basics — `01-folder`

Goal: Write your first React component and understand JSX syntax.

What is JSX?

JSX looks like HTML but lives inside JavaScript. React turns it into real DOM elements.

```
const App = () => {
  return (
    <>
      <div id="parent">
        <h1 id="child1">App1</h1>
        <h2 id="child2">Anmol</h2>
      </div>
      <div id="parent2">
        <h1 id="child1">App2</h1>
        <h2 id="child2">Anmol</h2>
      </div>
    </>
  )
}

export default App
```

Key concepts

Concept	Explanation
Component	A function that returns JSX
Arrow function	<code>const App = () => { ... }</code> — modern way to define components
Fragment <code><>...</></code>	Wraps multiple elements without adding an extra <code><div></code> to the DOM
export default	Makes the component importable in other files
className	React's version of HTML <code>class</code> (because <code>class</code> is a JS reserved word)
id	Works the same as in HTML

Rules of JSX

1. Return **one parent element** (or use a Fragment).
2. All tags must be **closed** — `<br />` not `<br>`.
3. Use `className` instead of `class`.
4. JavaScript expressions go inside `{ curly braces }`.

3. Components — 02-components

Goal: Split UI into reusable, separate files.

File structure

```
src/  
├─ App.jsx  
└─ components/  
    └─ Card.jsx
```

Parent uses the child component

```
// App.jsx  
import Card from './components/Card'  
  
const App = () => {  
  return (  
    <div className='parent'>  
      <Card />  
      <Card />  
    </div>  
  )  
}
```

Child component with local variables

```
// Card.jsx  
function Card() {  
  const age = 30  
  const name = 'anmol'  
  
  return (  
    <div className="card">  
      <h1>My name is {name}</h1>  
      <p>I am {age} years old</p>  
    </div>  
  )  
}  
  
export default Card
```

Key concepts

Concept	Explanation
Import	<code>import Card from './components/Card'</code> — brings in another file
Reuse	<code><Card /></code> used twice renders two separate cards
Variables in JSX	<code>{name}</code> and <code>{age}</code> insert JS values into the markup
Component naming	Always start with a capital letter (<code>Card</code> , not <code>card</code>)

Why components?

- **Reusability** — write once, use anywhere
- **Organization** — each piece of UI lives in its own file
- **Maintainability** — change `Card.jsx` once, all instances update

4. Props & Lists — 03-props

Goal: Pass data from parent to child and render lists with `.map()`.

What are props?

Props (properties) are how a parent component sends data to a child. They are **read-only** — a child cannot change its own props.

Data array in the parent

```
const jobs = [  
  {  
    company: "Amazon",  
    posted: "5 days ago",  
    title: "UX / UI Designer",  
    level: "Senior Level",  
    salary: "$120/hr",  
    location: "Mumbai, India"  
  },  
  // ... more jobs  
]
```

Passing props + rendering a list

```
const App = () => {
  return (
    <div className='parentcard'>
      {jobs.map(function(arrval, idx) {
        return (
          <div key={idx}>
            <Card
              company={arrval.company}
              posted={arrval.posted}
              title={arrval.title}
              level={arrval.level}
              salary={arrval.salary}
              location={arrval.location}
            />
          </div>
        )
      })}
    </div>
  )
}
```

Receiving props in the child

```
const Card = (props) => {
  return (
    <div className='card'>
      <h2>{props.companyname} <span>{props.posted}</span></h2>
      <h3>{props.title}</h3>
      <span className='label'>{props.level}</span>
      <h3 className='price'>{props.salary}</h3>
      <small>{props.location}</small>
    </div>
  )
}
```

Using third-party icons

```
import { Save, StickyNote } from 'lucide-react'

<button>Save <Save size={14} /></button>
<button>Apply <StickyNote size={12} /></button>
```

Key concepts

Concept	Explanation
<code>props</code>	Object containing all data passed from parent
<code>props.title</code>	Access a specific prop by name
<code>.map()</code>	Loops over an array and returns JSX for each item
<code>key</code>	Unique identifier React needs for list items (<code>key={idx}</code>)
Destructuring (alternative)	<code>const Card = ({ title, company }) => ...</code>

Important: prop names must match

Parent passes `company={...}` → child must read `props.company` , not `props.companyname` .

```
// Parent
<Card company="Amazon" />

// Child – correct
<h2>{props.company}</h2>

// Child – wrong (undefined)
<h2>{props.companyname}</h2>
```

What you built

A job listing board with 10 cards (Amazon, Google, Microsoft, Adobe, Netflix, Spotify, Airbnb, Meta, Apple, Figma) — each showing company, title, level, salary, and location.

5. CSS Modules — 05-css

Goal: Scope styles to individual components so class names never clash.

File structure

```
src/components/  
├─ header/  
│   ├── Header.jsx  
│   └─ header.module.css  
└─ button/  
    ├── Button.jsx  
    └─ button.module.css
```

How CSS Modules work

```
// Header.jsx  
import styles from './header.module.css'  
  
const Header = () => {  
  return (  
    <div className={styles.header}>  
      <h3>Logo</h3>  
      <button className={styles.btn}>Menu</button>  
    </div>  
  )  
}
```

```
/* header.module.css */  
.header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 5px 30px;  
  background: rgb(54, 126, 185);  
  border-radius: 8px;  
}  
  
.btn {  
  background: yellow;  
  padding: 10px 20px;  
  border-radius: 12px;  
}
```

```
// Button.jsx – also has a .btn class, but no conflict
import styles from './button.module.css'

const Button = () => {
  return <button className={styles.btn}>Login Me!</button>
}
```

Key concepts

Concept	Explanation
<code>.module.css</code>	Special file extension that enables scoping
<code>import styles from '...'</code>	Imports an object of scoped class names
<code>styles.header</code>	Applies the scoped version of <code>.header</code>
No clashes	<code>.btn</code> in Header and <code>.btn</code> in Button are completely separate

CSS Modules vs regular CSS

Regular CSS	CSS Modules
<code>className="header"</code>	<code>className={styles.header}</code>
Global — affects entire app	Scoped — only affects this component
Class names can conflict	Auto-generated unique names prevent conflicts

6. Tailwind CSS — [06-tailwind](#)

Goal: Style components using utility classes instead of writing custom CSS.

Setup

`vite.config.js`

```
import tailwindcss from '@tailwindcss/vite'

export default {
  plugins: [react(), tailwindcss()],
}
```

index.css

```
@import "tailwindcss";
```

Using utility classes

```
const App = () => {  
  return (  
    <div className='text-3xl text-center font-bold m-5 p-10 rounded-3xl bg-gray'>  
      App  
    </div>  
  )  
}
```

Common Tailwind classes

Class	CSS equivalent
text-3xl	font-size: 1.875rem
font-bold	font-weight: 700
text-center	text-align: center
m-5	margin: 1.25rem
p-10	padding: 2.5rem
rounded-3xl	border-radius: 1.5rem
flex	display: flex
justify-between	justify-content: space-between
items-center	align-items: center
w-full	width: 100%
h-full	height: 100%
gap-7	gap: 1.75rem
overflow-x-auto	horizontal scroll when content overflows

Key idea

Instead of writing CSS in a separate file, you compose styles directly in `className` using small, single-purpose utility classes.

7. UI Project — 07-ui-project

Goal: Combine everything — nested components, props, Tailwind, icons, and dynamic styling — into one full page.

Component tree

```

App
├── Topsection
│   ├── Topleft      → "Target Audience" badge
│   └── Topright     → "Digital Banking Platform" badge
└── Bottomsection    → receives users[] prop
    ├── Bottomleft   → heading + MoveUpRight icon
    └── Bottomright  → maps users → Cards
        └── Cards     → profile card (image, tag, intro)
  
```

Data lives in the parent, flows down

```

// App.jsx
const users = [
  { img: "...", intro: "...", color: "blue", tag: "Satisfied" },
  { img: "...", intro: "...", color: "lightgreen", tag: "Happy" },
  // ... 10 users total
]

return (
  <div>
    <Topsection />
    <Bottomsection users={users} />
  </div>
)
  
```

Props passed through multiple levels

```
// Bottomsection.jsx – receives users, passes to Bottomright
const Bottomsection = (props) => {
  return (
    <div className="flex justify-between items-center w-full h-[85vh]">
      <Bottomleft />
      <Bottomright users={props.users} />
    </div>
  )
}

// Bottomright.jsx – maps array to Cards
const Bottomright = (props) => {
  return (
    <div className="flex flex-nowrap overflow-x-auto p-5 w-[70%] h-full gap-7">
      {props.users.map(function(elem, idx) {
        return (
          <Cards
            key={idx}
            id={idx}
            image={elem.img}
            intro={elem.intro}
            color={elem.color}
            tag={elem.tag}
          />
        )
      })}
    </div>
  )
}
```

Dynamic inline styles

When a value comes from data (not a fixed class), use the `style` prop:

```
<div style={{ backgroundColor: props.color }}>
  {props.id + 1}
</div>

<button style={{ backgroundColor: props.color }}>
  {props.tag}
</button>
```

Card layout techniques used

Technique	Where used
<code>flex + justify-between</code>	Top and bottom section layout
<code>overflow-x-auto + flex-nowrap</code>	Horizontal scrolling card row
<code>relative + absolute</code>	Overlay text and buttons on card images
<code>object-cover</code>	Image fills card without distortion
<code>shrink-0</code>	Cards don't shrink below their set width
<code>h-[85vh]</code>	Section takes 85% of viewport height

What you built

A "Prospective customer segmentation" page for a digital banking platform — left side has a heading, right side has 10 horizontally scrollable profile cards with photos, colored tags, and intro text.

8. Event Handlers — `08-functions`

Goal: Make the UI respond to user actions — clicks, typing, scrolling.

Named handler functions

```
const App = () => {
  function btnClicked() {
    console.log('Button Clicked')
  }

  function checkInputVal(val) {
    console.log(val)
  }

  return (
    <div>
      <button onClick={btnClicked} className='btn'>
        Click Me
      </button>

      <input
        type='text'
        onChange={function(elem) { checkInputVal(elem.target.value) }}
        placeholder='Enter Name'
      />
    </div>
  )
}
```

Inline handler (anonymous function)

```
<div onWheel={function(elem) { console.log(elem.deltaY) }}>
  ...
</div>
```

Event handlers covered

Event	Fires when	Access data via
<code>onClick</code>	Element is clicked	—
<code>onChange</code>	Input value changes	<code>elem.target.value</code>
<code>onWheel</code>	Mouse wheel scrolls	<code>elem.deltaY</code> (scroll direction/speed)

Critical rule: function reference vs function call

```
// Correct – passes the function (React calls it on click)
<button onClick={btnClicked}>

// Wrong – calls immediately on every render
<button onClick={btnClicked()}>

// Correct – inline arrow/anonymous function
<input onChange={(elem) => checkInputVal(elem.target.value)} />
```

How events flow

- 1. User performs an action (click, type, scroll).
- 2. React calls the function you attached to the event prop.
- 3. Your function runs — update state, log to console, call an API, etc.

9. Quick Reference

Module summary

Module	Topic	What you built
01-folder	JSX, Fragments	Two parent divs with headings
02-components	Reusable components	Two Card components with name/age
03-props	Props, <code>.map()</code> , icons	10 job listing cards
05-css	CSS Modules	Header + Login button
06-tailwind	Tailwind utilities	Styled div with utility classes
07-ui-project	Full page, component tree	Customer segmentation scroll gallery
08-functions	Event handlers	Click, input, and scroll events

Concepts progression

JSX syntax

- └ Components (split into files)
 - └ Props (pass data parent → child)
 - └ Lists (.map + key)
 - └ CSS Modules (scoped styles)
 - └ Tailwind (utility classes)
 - └ Event handlers (onClick, onChange)

Syntax cheat sheet

```
// Component
const MyComponent = () => {
  return <div>Hello</div>
}

// Import component
import MyComponent from './components/MyComponent'

// Variables in JSX
<p>{variableName}</p>

// Props
<Child name="Anmol" age={30} />
const Child = (props) => <p>{props.name}</p>

// List rendering
{items.map((item, idx) => <Card key={idx} data={item} />)}

// CSS Modules
import styles from './file.module.css'
<div className={styles.container}>

// Tailwind
<div className="flex justify-center p-4 rounded-lg">

// Inline dynamic style
<div style={{ backgroundColor: color }}>

// Events
<button onClick={handleClick}>
<input onChange={(e) => handleChange(e.target.value)}>

// Icons
import { IconName } from 'lucide-react'
<IconName size={14} />
```

Common mistakes to avoid

Mistake	Fix
<code>class="header"</code>	Use <code>className="header"</code>
<code>onClick={myFunc()}</code>	Use <code>onClick={myFunc}</code>
Missing <code>key</code> in <code>.map()</code>	Add <code>key={idx}</code> or <code>key={item.id}</code>
Prop name mismatch	Parent <code>company=</code> → child <code>props.company</code>
Multiple root elements without wrapper	Wrap in <code><...></code> or a <code><div></code>

What's next (not covered yet)

- **useState** — data that changes over time
- **Conditional rendering** — show/hide UI based on state
- **Forms** — controlled inputs tied to state
- **useEffect** — run code on mount or when data changes
- **Fetching data** — API calls inside components

Last updated: July 2026 · Project: reactpractice